

Praktikum Einführung in die Robotik

Einführung ROS2 und Igus Rebel

Niklas Noack¹, Jan-Luca Regenhardt¹, Ben Schuhknecht¹

¹Autonomous Mobile Robotics Lab
Hochschule für angewandte Wissenschaft und Kunst

30. April 2026

Wiederholung

Gazebo

Igus Rebel simulieren

Igus Rebel steuern

RViz

RViz nutzen

Robotersteuerung mit
Python

ROS 2 Workspace

Arbeitsumgebung

Pakete

Nützliche Links

Wiederholung

Gazebo

Igus Rebel simulieren

Igus Rebel steuern

RViz

RViz nutzen

Robotersteuerung mit Python

ROS 2 Workspace

Arbeitsumgebung

Pakete

Nützliche Links

Wiederholung

Gazebo

Igus Rebel simulieren

Igus Rebel steuern

RViz

RViz nutzen

Robotersteuerung mit
Python

ROS 2 Workspace

Arbeitsumgebung

Pakete

Nützliche Links

- ▶ **Nachrichten** werden durch Netzwerke von Knoten **Nodes** ausgetauscht
- ▶ Eine dezentrale **Middleware** koordiniert Kommunikation zwischen Knoten
- ▶ Asynchrone Nachrichten werden über **Topics** (**Publisher** und **Subscriber**) gesendet
- ▶ Synchrone Nachrichten werden durch **Services** (**Request** und **Response**) gesendet

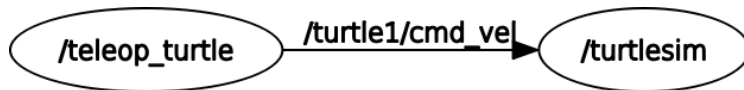


Abbildung: *Publisher, Topic, Subscriber*

Wiederholung 2/2

Wichtige Befehle:

- ▶ Strg + Alt + T zum öffnen eines Terminals
- ▶ `ros2 run Paketname Knotenname` zum starten eines *nodes*
Bsp.: `ros2 run turtlesim draw_square`

Hilfe über Parameter `--h`; Bsp.: `ros2 node --h` führt

zu:

- ▶ `ros2 node list` listet alle *nodes* auf
- ▶ `ros2 node info` gibt Info über einen *node* aus

Befehle für Nachrichten (Topics) zwischen Nodes

- ▶ `ros2 topic list` listet topics auf
- ▶ `ros2 topic echo /topic_name` „lauscht“ einem topic
- ▶ `ros2 topic type /topic_name` gibt Datentyp des topics an

Autovervollständigung

- ▶ Autovervollständigung mit [Tab] (Tabulator) Bsp.:
- ▶ `ros [Tab]` listet alle möglichen Befehle beginnend mit `ros2` auf.
- ▶ `ros2 node [Tab]` Listet alle möglichen Parameter auf.

Wiederholung

Gazebo

Igus Rebel simulieren
Igus Rebel steuern

RViz

RViz nutzen
Robotersteuerung mit
Python

ROS 2 Workspace

Arbeitsumgebung
Pakete

Nützliche Links

Was ist Gazebo?

- ▶ 3D-Simulator für Roboter und Umgebungen
- ▶ Realistische Physik (Kollision, Gravitation, Bewegung)
- ▶ Simuliert typische Robotersensoren:
 - ▶ Laser-Scan (LiDAR)
 - ▶ Kamera / Tiefenkamera
 - ▶ IMU, Odometrie
 - ▶ inkl. Fehlern (Rauschen, Nullpunktfehler, etc..)
- ▶ Grafische Oberfläche zum Beobachten und Steuern
 - ▶ Erlaubt das Testen von Robotern, ohne echte Hardware
 - ▶ Nutzt ROS 2-Topics, Parameter und Dienste



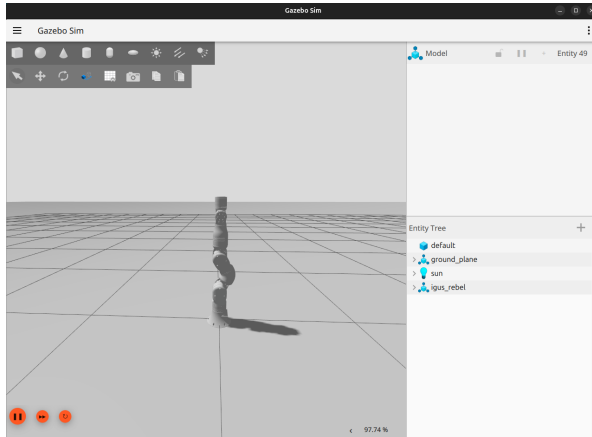
GAZEBO

Abbildung: Gazebo
Logo

Simulation in Gazebo

Ausführen im Terminal

- ▶ `igus`
- ▶ `ros2 launch igus_rebel_moveit_config`
`igus_rebel_simulated.launch.py`



RViz: Visualisierung von Robotern und Sensoren

Was man in RViz sehen kann:

- ▶ **Sensoren:**
Laserscanner,
Kameras,
Punktwolken
- ▶ **Robotermodelle:**
Pose, TF, Rebel
- ▶ **Navigation,
Kinematik:**
geplante Pfade,
Ziele, Goal Pose
- ▶ Live-Daten visualisieren, Pfade und Ziele prüfen

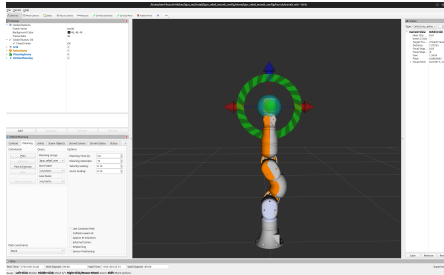


Abbildung: Igus Rebel mit
Motionplanning

Wiederholung

Gazebo

Igus Rebel simulieren

Igus Rebel steuern

RViz

RViz nutzen

Robotersteuerung mit
Python

ROS 2 Workspace

Arbeitsumgebung

Pakete

Nützliche Links

RViz nutzen

- ▶ Über **MotionPlanning** Igus steuern
- ▶ **TF** Link6-Position auslesen um später Befehle zu erzeugen

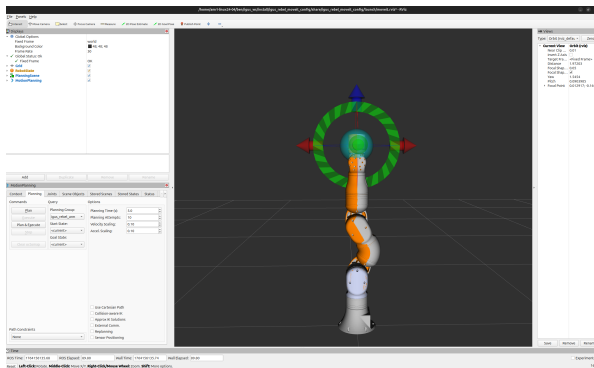


Abbildung: RViz - Igus Rebel

Steuern des Igus Rebel mit Python

- ▶ Strg+Alt+T um Terminal zu öffnen.
- ▶ `ros2 run igus_student simple_control`

```
pc@pc-MS-7C79: ~
pc@pc-MS-7C79: ~ 128x41
pc@pc-MS-7C79:~$ ros2 run igus_student simple_control
-----SIMPLE IGUS REBEL CONTROL-----
[INFO] [1764178291.528987141]: [simple_robot_controller]: Connecting to MoveGroup...
[INFO] [1764178291.771688291]: [simple_robot_controller]: Connected to MoveGroup
[INFO] [1764178291.772364459]: [simple_robot_controller]: Added collision object: table
[INFO] [1764178292.274847298]: [simple_robot_controller]: Loaded 1 collision objects
Starting motion sequence...
[INFO] [1764178292.274324248]: [simple_robot_controller]: Target: (0.488, 0.888, 0.388)
[INFO] [1764178292.275115323]: [simple_robot_controller]: Sending goal to MoveGroup...
[INFO] [1764178292.282968352]: [simple_robot_controller]: Goal accepted, executing...
[INFO] [1764178292.283311475]: [simple_robot_controller]: Waiting for result...
[INFO] [1764178292.778666913]: [simple_robot_controller]: Movement successful
[INFO] [1764178293.779380334]: [simple_robot_controller]: Waiting for robot to settle...
[INFO] [1764178296.792168929]: [simple_robot_controller]: Robot settled (vel=0.888)
Reached home position
[INFO] [1764178296.792466382]: [simple_robot_controller]: Target: (0.488, 0.288, 0.388)
[INFO] [1764178296.793171296]: [simple_robot_controller]: Sending goal to MoveGroup...
[INFO] [1764178296.794384731]: [simple_robot_controller]: Goal accepted, executing...
[INFO] [1764178296.794929298]: [simple_robot_controller]: Waiting for result...
[INFO] [1764178296.104628783]: [simple_robot_controller]: Movement successful
[INFO] [1764178296.104444726]: [simple_robot_controller]: Waiting for robot to settle...
[INFO] [1764178299.189756421]: [simple_robot_controller]: Robot settled (vel=0.888)
Reached position 1
[INFO] [1764178299.198852276]: [simple_robot_controller]: Target: (0.488, -0.288, 0.388)
[INFO] [1764178299.198618391]: [simple_robot_controller]: Sending goal to MoveGroup...
[INFO] [1764178299.191681791]: [simple_robot_controller]: Goal accepted, executing...
[INFO] [1764178299.191967128]: [simple_robot_controller]: Waiting for result...
[INFO] [1764178301.632358956]: [simple_robot_controller]: Movement successful
[INFO] [1764178301.632762886]: [simple_robot_controller]: Waiting for robot to settle...
[INFO] [1764178302.638126164]: [simple_robot_controller]: Robot settled (vel=0.888)
Reached position 2
[INFO] [1764178302.638426816]: [simple_robot_controller]: Target: (0.488, 0.888, 0.388)
[INFO] [1764178302.638997934]: [simple_robot_controller]: Sending goal to MoveGroup...
[INFO] [1764178302.639979794]: [simple_robot_controller]: Goal accepted, executing...
[INFO] [1764178302.648336223]: [simple_robot_controller]: Waiting for result...
[INFO] [1764178304.138085697]: [simple_robot_controller]: Movement successful
[INFO] [1764178304.138475428]: [simple_robot_controller]: Waiting for robot to settle...
[INFO] [1764178305.137235784]: [simple_robot_controller]: Robot settled (vel=0.888)
Returned home
-----PROGRAM COMPLETED-----
```

Abbildung: simple_control

- Igus Rebel simulieren
- Igus Rebel steuern

RViz nutzen
Robotersteuerung mit
Python

Arbeitsumgebung
Pakete

Nützliche Links

```
# -----
# YOUR PROGRAM HERE
# -----

print("Starting motion sequence...")

# Move to home - now automatically waits for robot to settle
move_home()
print("Reached home position")

# Move to first position - automatically waits
safe_move_to_pose(0.4, 0.2, 0.3, pi, 0.0, 0.0)
print("Reached position 1")

# Move to second position - automatically waits
safe_move_to_pose(0.4, -0.2, 0.3, pi, 0.0, 0.0)
print("Reached position 2")

safe_move_to_pose(0.4, 0.0, 0.1, pi, 0.0, 0.0)
print("Reached position 3")

# Return home - automatically waits
move_home()
print("Returned home")

# -----

print("-" * 20 + " PROGRAM COMPLETED " + "-" * 20)
```

- ▶ Anweisungen nach Belieben anpassen
- ▶ `cd /home/ubuntu/igus_ws`
- ▶ `colcon build --packages-select igus_student`
- ▶ `ros2 run igus_student simple_control`

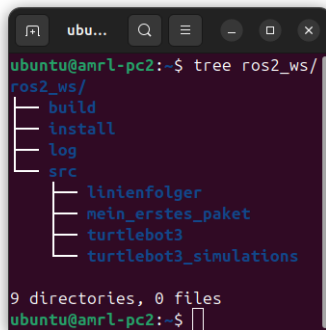
Arbeitsumgebung

Der *ROS 2-Workspace* stellt den persönlichen Arbeitsbereich dar. Er befindet sich typischerweise im Heimverzeichnis:

`/home/benutzer/ros2_ws` bzw. `~/ros2_ws`

► ROS 2 Workspace:

- `src` → enthält alle eigenen Pakete
- `build` → wird beim Kompilieren erzeugt
- `install` → enthält die installierten, ausführbaren Nodes
- `log` → Debug- und Laufzeitlogs

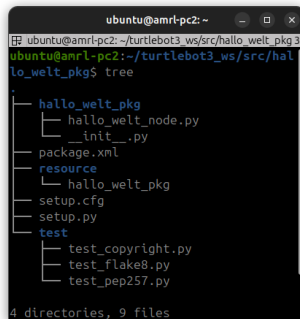


```
ubuntu@amrl-pc2:~$ tree ros2_ws/  
ros2_ws/  
├── build  
├── install  
├── log  
└── src  
    ├── linienfolger  
    ├── mein_erstes_paket  
    ├── turtlebot3  
    └── turtlebot3_simulations  
  
9 directories, 0 files  
ubuntu@amrl-pc2:~$
```

Wichtig: Alle eigenen ROS 2-Pakete liegen im `src`-Ordner.

Abbildung: ROS 2 *Workspace* Struktur

- ▶ Stellt kleinstmögliche Organisationseinheit dar
- ▶ vergleichbar mit Linux *Software-Paketen*
- ▶ Beinhaltet:
 - ▶ *Nodes*
 - ▶ Programme (Skripte,)
 - ▶ Bibliotheken
 - ▶ Konfigurationsdateien



```
ubuntu@amrl-pc2: ~  
ubuntu@amrl-pc2: ~/turtlebot3_ws/src/hallo_welt_pkg$ tree  
.  
├── hallo_welt_pkg  
│   ├── hallo_welt_node.py  
│   ├── __init__.py  
│   └── package.xml  
├── resource  
│   └── hallo_welt_pkg  
├── setup.cfg  
├── setup.py  
└── test  
    ├── test_copyright.py  
    ├── test_flake8.py  
    └── test_pep257.py  
4 directories, 9 files
```

Abbildung: Paket: Beispielprojekt

- ▶ *<paketname>* beinhaltet *.py-Skripte mit Node-Funktion
- ▶ package.xml - *Softwareabhängigkeiten*, *Copyright*, *Autor*, etc.
- ▶ *resources* beinhaltet 0 kB große *<paketname>* „Markerdatei“
- ▶ setup.py - Bauanleitung für *Python*
- ▶ *launch* beinhaltet *.launch.py-Dateien

Nützliche Links 1

Tutorium Robotik

N. Noack,
J.-L. Regenhardt,
B. Schuhknecht

RViz nutzen

Robotersteuerung mit Python

Arbeitsumgebung

Pakete

Nützliche Links

- ## ► ROS2-Framework (Jazzy)

- ▶ <https://docs.ros.org/en/jazzy/>

- ROS2-Tutorials (Jazzy)

<https://docs.ros.org/en/jazzy/Tutorials.html>

- ## ► ROS2-Befehle / CLI (Jazzy)

<https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Using-ROS-2-CLI.html>

- ### ► TurtleBot3 (ROS2)

https://emanual.robotis.com/docs/en/platform/turtlebot3/ros2_overview/

- ## ▶ Linux *Basics*

<https://cheatography.com/davechild/cheat-sheets/linux-command-line/>

► **ros2 launch (Jazzy)**

<https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Launch/Creating-Launch-Files.html>

► **ROS2-Package erstellen (Jazzy)**

<https://docs.ros.org/en/jazzy/How-To-Guides/Developing-a-ROS-2-Package.html>

► **Publisher/Subscriber schreiben (C++) (Jazzy)**

<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Publisher-And-Subscriber.html>

Wiederholung

Gazebo

Igus Rebel simulieren

Igus Rebel steuern

RViz

RViz nutzen

Robotersteuerung mit
Python

ROS 2 Workspace

Arbeitsumgebung

Pakete

Nützliche Links

► Gazebo (ROS2 / Jazzy)

<https://docs.ros.org/en/jazzy/Tutorials/Advanced/Simulators/Gazebo/Gazebo.html>

► RViz2 (Jazzy)

<https://docs.ros.org/en/jazzy/Tutorials/Intermediate/RViz/RViz-Main.html>

► Composable Nodes & Launch (Jazzy)

<https://docs.ros.org/en/jazzy/How-To-Guides/Launching-composable-nodes.html>

► Build-System (colcon & ament, Jazzy)

<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Colcon-Tutorial.html>

Niklas Noack: niklas.noack@hawk.de

Jan-Luca Regenhardt: jan-luca.regenhardt@stud.hawk.de